Московский государственный университет

имени М. В. Ломоносова

Майстренко Марат Алексеевич

«Раскрытие проблемы организации пересылок MPI»

Москва, 2025

1.Постановка проблемы

В современных вычислительных системах эффективная организация пересылок данных играет ключевую роль для достижения высокой производительности. Одним из инструментов для межпроцессного взаимодействия является **MPI** (Message Passing Interface), который предоставляет набор функций для обмена сообщениями. Однако при использовании MPI могут возникать проблемы, связанные с неоптимальной организацией пересылок данных.

Последовательная пересылка данных между процессами вместо одновременного обмена приводит к значительному снижению масштабируемости. Это критично в задачах, где требуется интенсивный обмен данными между множеством процессов. Если вместо организации одновременных пересылок между парами процессов данные передаются последовательно по цепочке, это создаёт избыточную задержку и увеличивает общее время выполнения программы. Такие накладные расходы могут существенно ухудшить масштабируемость приложения, особенно при работе на большом количестве процессов.

2.Пример неправильного кода

Рассмотрим задачу вычисления интеграла $\int_0^{\pi} \sin(x)\, dx$, которая хорошо поддается распараллеливанию с помощью метода левых (нижних) прямоугольников. Участок интегрирования делится на P (количество процессов), далее каждый процесс выполняет подсчёт своего участка и отправляет соседнему процессу свою левую границу. При обмене используется логически верный последовательный порядок, сначала 1-ый отправляет второму, затем 2-ой принимает и начинает отправку 3-ему и т.д.. Это приведет к тому, что значительная часть времени выполнения программы будет потрачена не на полезные вычисления, а на синхронизацию пересылок.

<u>Основная проблема</u> — **последовательная пересылка** данных с помощью MPI средств. В программе используются блокирующие операции MPI_Send и MPI_Recv. Следовательно, каждый процесс:

1. Дожидается приема числа от другого процесса.
2. Добавляет в локальную сумму.
3. Отправляет и дожидается приема числа следующим процессом.

```
for (int i = 0; i < local_n; i++) local_sum += f(local_a + i * h) * h;

if (rank == 0) {
    double border_value = f(local_b) * h;
    MPI_Send(&border_value, 1, MPI_DOUBLE, rank + 1, 0, MPI_COMM_WORLD);
} else {
    double prev_border_value;
    MPI_Recv(&prev_border_value, 1, MPI_DOUBLE, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    local_sum += prev_border_value;
    double border_value = f(local_b) * h;
    if (rank < size - 1) MPI_Send(&border_value, 1, MPI_DOUBLE, rank + 1, 0, MPI_COMM_WORLD);

}

MPI_Allreduce(&local_sum, &total_sum, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);

return total_sum;
```

Последствия:

- **Накладные расходы**: Каждая синхронизация требует времени и ресурсов.
- **Снижение производительности**: Значительная часть времени выполнения тратится на синхронизацию пересылок, а не на полезные вычисления.

4. Исправление неправильного кода

Чтобы устранить проблему, нужно **организовать асинхронную отправку числа**. Это позволит организовать одновременный обмен.

```
for (int i = 0; i < local_n; i++) local_sum += f(local_a + i * h) * h;

MPI_Request send_request = MPI_REQUEST_NULL;
MPI_Request recv_request = MPI_REQUEST_NULL;
double border_value = 0.0, prev_border_value = 0.0;

if (rank > 0) MPI_Irecv(&prev_border_value, 1, MPI_DOUBLE, rank - 1, 0, MPI_COMM_WORLD, &recv_request);

if (rank < size - 1) {
    border_value = f(local_b) * h;
    MPI_Isend(&border_value, 1, MPI_DOUBLE, rank + 1, 0, MPI_COMM_WORLD, &send_request);
}

if (rank > 0) {
    MPI_Wait(&recv_request, MPI_STATUS_IGNORE);
    local_sum += prev_border_value;
}

if (rank < size - 1) MPI_Wait(&send_request, MPI_STATUS_IGNORE);

MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

return total_sum;
```

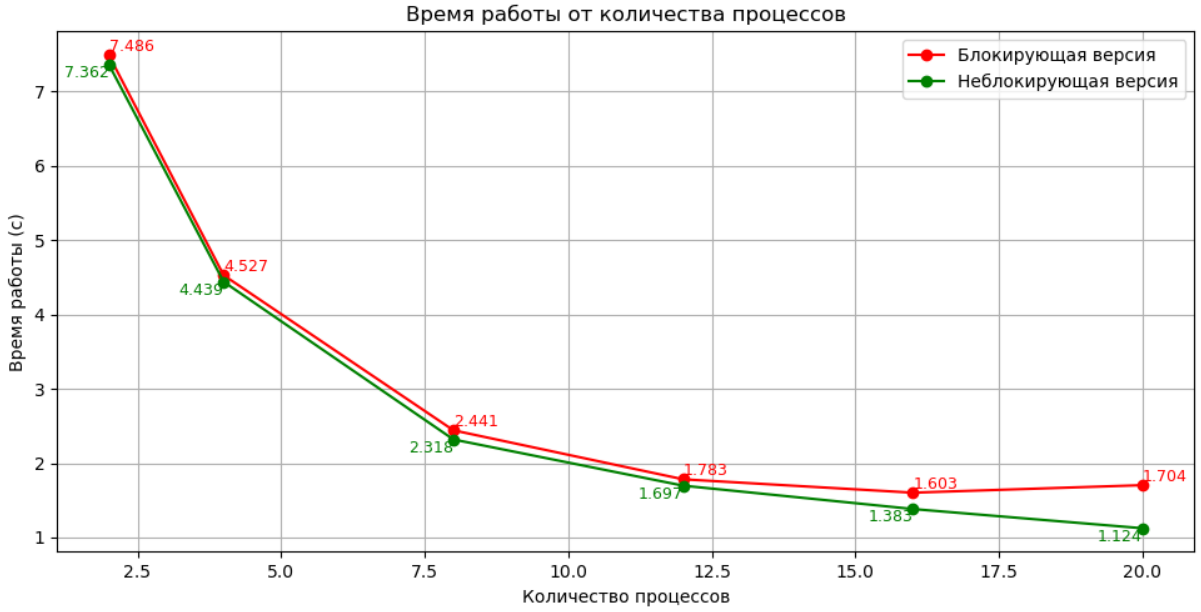
5. Измерение ускорения и анализ результатов

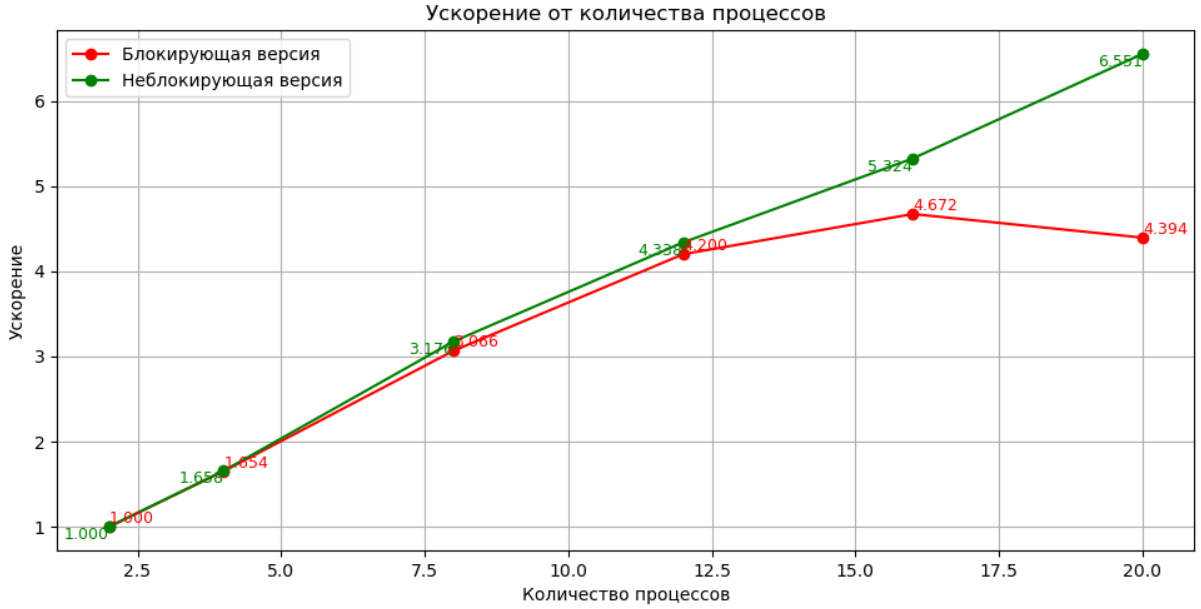
Неблокирующая версия (асинхронные операции):

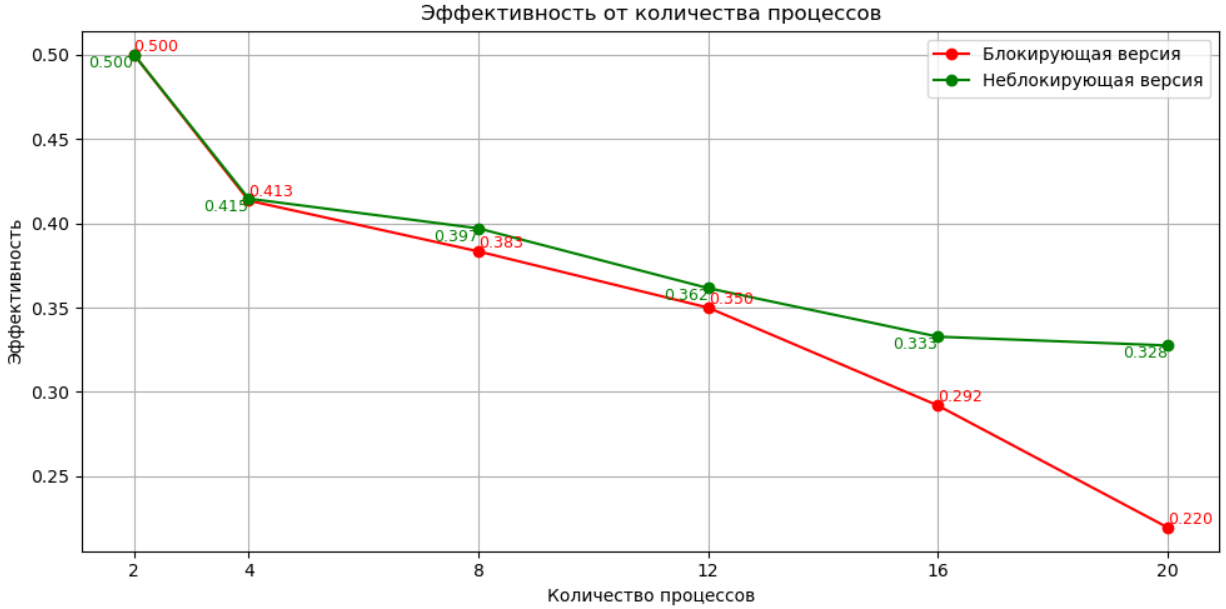
- *Лучшая масштабируемость*
- *Оптимальная производительность:* Наибольшее ускорение достигается при **20 процессах** (~1.124 сек)

Блокирующая версия (синхронные операции):

- *Хуже масштабируемость*
- *Рост времени из-за накладных расходов*







Замеры производились на вычислительном комплексе **IBM Polus.**

Процессы	**Блокирующая версия [с.]**	**Неблокирующая версия [с.]**
2	7.486442	7.361872
4	4.527006	4.439419
8	2.441433	2.318302
12	1.782583	1.696897
16	1.602507	1.382669
20	1.703662	1.123752

6. Профилирование программы с помощью perf

Для анализа производительности был использован инструмент perf record с параметрами -F 1000 (частота сэмплирования 1000 Гц) и -g (сбор цепочек вызовов). Основное время выполнения (97.83%) приходится на работу с MPI-коммуникациями, при этом значительная часть ресурсов тратится на системные вызовы и синхронизацию потоков. Наибольшие накладные расходы связаны с коллективными операциями и обработкой сообщений в реализации IBM Spectrum MPI.

Функция blocking_version занимает 61.68% времени выполнения, из которых 59.46% приходится на ожидание завершения операций в PAMI_Context_advancev. Высокий процент времени, проведенного в pthread_spin_lock (13.40%), указывает на активную конкуренцию потоков за ресурсы. При этом лишь 44.47% процессорного времени используется непосредственно для вычислений, что свидетельствует о неоптимальном соотношении работы и ожидания.

Функция non_blocking_version демонстрирует лучшее распределение времени (34.85% общего времени), однако также страдает от накладных расходов на вызовы MPI_Reduce (23.78%) и работу с памятью. Относительно низкая доля математических операций (5.42% для sin) подтверждает, что основным узким местом являются коммуникационные задержки, а не вычислительная сложность алгоритмов.

```
Samples: 25K of event 'cycles:ppp', Event count (approx.): 107355593312
  Children      Self  Command   Shared Object                    Symbol
+   97.83%     0.00%  zad_2     [unknown]                        [.] 0000000000000000
+   97.75%     0.00%  zad_2     libc-2.17.so                     [.] __libc_start_main
+   97.75%     0.00%  zad_2     libc-2.17.so                     [.] generic_start_main.isra.0
+   97.70%     0.16%  zad_2     zad_2                            [.] main
+   61.68%     0.79%  zad_2     zad_2                            [.] blocking_version
+   59.46%    12.02%  zad_2     libpami.so                       [.] PAMI_Context_advancev
+   52.49%     1.16%  zad_2     libcollectives.so                [.] LIBCOLL_Advance
+   51.87%     0.32%  zad_2     libmpi_ibm.so.2.0.1              [.] PMPI_Allreduce
+   51.25%     4.59%  zad_2     mca_coll_ibm.so                  [.] mca_coll_ibm_allreduce
+   34.85%     0.88%  zad_2     zad_2                            [.] non_blocking_version
+   26.26%     2.54%  zad_2     libpami.so                       [.] PAMI::Device::IBV::Device::advance
+   24.10%     0.26%  zad_2     libmpi_ibm.so.2.0.1              [.] PMPI_Reduce
+   23.78%     1.34%  zad_2     mca_coll_ibm.so                  [.] mca_coll_ibm_reduce
+   19.22%     1.29%  zad_2     libcollectives.so                [.] LibColl::NativeInterfaceP2P<true, false>::call_work_fn
+   13.40%    13.38%  zad_2     libpthread-2.17.so               [.] pthread_spin_lock
+   12.91%     0.00%  zad_2     libmlx5.so.1.0.0                 [.] 0xffffefffe9db9a8
+   10.76%     1.88%  zad_2     libcollectives.so                [.] CCMI::Executor::HybridAllreduceT<CCMI::ConnectionManager::RankBasedConnMgr>::advance_fn
+   10.34%     0.13%  zad_2     libcollectives.so                [.] LIBCOLL_AutoSelect
+   10.06%     0.35%  zad_2     libcollectives.so                [.] LibColl::Adapter::StaticCollSelAdviser::autoSelect
+    6.95%     0.26%  zad_2     libopen-pal.so.2.1.0             [.] opal_progress
+    6.72%     0.41%  zad_2     libcollectives.so                [.] CCMI::Protocols::Allreduce::AsyncAllreduceFactoryT<CCMI::Protocols::Allreduce::HybridAsyncReduceT<CCMI::Conn
+    6.66%     6.65%  zad_2     libcollectives.so                [.] CCMI::Executor::ShmemShortBroadcastT<false, CCMI::Executor::ShmemAtomicBarrier>::advance_impl
+    5.42%     5.42%  zad_2     libm-2.17.so                     [.] __sin
+    5.34%     0.17%  zad_2     libmpi_ibm.so.2.0.1              [.] PMPI_Wait
+    5.17%     0.22%  zad_2     libmpi_ibm.so.2.0.1              [.] ompi_request_default_wait
+    5.13%     0.00%  zad_2     libmlx5.so.1.0.0                 [.] 0xffffefffe9bb9a8
+    4.73%     0.04%  zad_2     libmpi_ibm.so.2.0.1              [.] PMPI_Recv
+    4.68%     0.13%  zad_2     mca_pml_pami.so                  [.] mca_pml_pami_recv
+    4.44%     0.40%  zad_2     zad_2                            [.] f
+    4.39%     3.81%  zad_2     libcollectives.so                [.] CCMI::Executor::ShmemBarrier::advance_impl
+    4.22%     0.02%  zad_2     mca_pml_pami.so                  [.] mca_pml_pami_progress
+    4.21%     0.12%  zad_2     mca_pml_pami.so                  [.] mca_pml_pami_progress_wait
+    3.63%     0.83%  zad_2     libcollectives.so                [.] CCMI::Executor::HybridReduceT<CCMI::ConnectionManager::CommSeqConnMgr>::advance_fn
+    3.13%     0.16%  zad_2     libpami.so                       [.] PAMI_Send_immediate
+    2.82%     2.82%  zad_2     libcollectives.so                [.] CCMI::Executor::ShmemReduceT<CCMI::Executor::ShmemAtomicBarrier>::advance_impl
+    2.56%     0.00%  zad_2     libmlx5.so.1.0.0                 [.] 0xffffefffe9dbdc0
+    2.19%     0.08%  zad_2     libpami.so                       [.] PAMI::Protocol::Send::Eager<PAMI::Device::Shmem::PacketModel<PAMI::Device::ShmemDevice<PAMI::Fifo::LinearFif
+    2.04%     0.09%  zad_2     libcollectives.so                [.] CCMI::Executor::HybridReduceT<CCMI::ConnectionManager::CommSeqConnMgr>::reset
+    2.00%     0.07%  zad_2     libcollectives.so                [.] CCMI::Protocols::Allreduce::ProtocolFactoryT<CCMI::Protocols::Allreduce::HybridOATCompositeT<CCMI::Executor:
+    1.92%     0.06%  zad_2     libcollectives.so                [.] CCMI::Executor::Allreduce<CCMI::ConnectionManager::CommSeqConnMgr>::reset
+    1.90%     0.21%  zad_2     libcollectives.so                [.] CCMI::Executor::ShmemAtomicBarrier::setTopology_impl
+    1.86%     0.38%  zad_2     libevent-2.0.so.5.1.10           [.] event_base_loop
```

7. Профилирование программы с помощью mpiP

Программа демонстрирует значительную, но умеренно сбалансированную зависимость от MPI-коммуникаций. В среднем **50.44%** времени выполнения (28.5 из 56.6 секунд) затрачивается на обмен данными между процессами. Это демонстрирует дисбаланс между вычислениями и коммуникациями. Наблюдается равномерное распределение нагрузки (вариация 84.19-88.60% между процессами), что подтверждает системный характер проблемы.

Основным узким местом остается операция Allreduce, занимающая 38.46% общего времени. При частоте вызовов 1000000 и общей продолжительности 12 секунд эта операция требует особого внимания для оптимизации. Существенный вклад также вносят операции Recv (25.51%) и Reduce (20.23%), что характерно для алгоритмов с интенсивным обменом данными.

Point-to-point операции (Send/Isend/Recv/Irecv) суммарно занимают 30.61% времени

@--- MPI Time (seconds)

Task	AppTime	MPITime	MPI%
0	2.83	1.84	64.93
1	2.83	1.73	61.36
2	2.83	1.55	54.82
3	2.83	1.56	54.98
4	2.83	1.61	56.81
5	2.83	1.56	55.11
6	2.83	1.5	52.95
7	2.83	1.48	52.17
8	2.83	1.55	54.80
9	2.83	1.53	53.95
10	2.83	1.57	55.55
11	2.83	1.51	53.35
12	2.83	1.56	55.19
13	2.83	1.53	53.97
14	2.83	1.57	55.37
15	2.83	1.2	42.25
16	2.83	0.87	30.75
17	2.83	0.865	30.61
18	2.83	0.901	31.84
19	2.83	1.08	38.02
*	56.6	28.5	50.44

@--- Aggregate Time (top twenty, descending, milliseconds)

Call	Site	Time	App%	MPI%	Count	COV
Allreduce	3	1.2e+04	21.25	42.12	1000000	0.55
Recv	10	7.46e+03	13.18	26.13	950000	0.58
Reduce	11	6.36e+03	11.25	22.30	950000	0.44
Wait	9	1.14e+03	2.01	3.99	950000	1.58
Reduce	4	607	1.07	2.13	50000	0.00
Send	8	293	0.52	1.03	900000	0.13
Isend	7	282	0.50	0.99	950000	0.14
Irecv	12	222	0.39	0.78	950000	0.02
Wait	2	118	0.21	0.41	950000	0.04
Send	5	18.5	0.03	0.06	50000	0.00
Barrier	1	14.7	0.03	0.05	20	0.79
Barrier	6	0.168	0.00	0.00	20	0.15

8. Заключение

Исследование наглядно демонстрирует важность правильной организации обменов с помощью MPI в параллельных вычислениях. Последовательный, блокирующий обмен может быть более понятным и логичным для человека, но он требует точно больших вычислительных и временных затрат чем неблокирующий.

Блокирующая версия наглядно демонстрирует, что накладные расходы на синхронизацию обмена могут значительно превышать выигрыш от параллельного выполнения задач. Неблокирующая версия явно показывает, как можно избежать лишних временных затрат при вычислении. Это указывает на необходимость тщательного анализа и оптимизации организации обмена между процесса средствами MPI.